

ANNEE UNIVERSITAIRE 2025-2026

RAPPORT DE PROJET DE FIN D'ANNEE
(PFA)

**Conception et développement d'un portail
fournisseur selon les normes de sécurité de la
DGSSI**

*Portail Fournisseur : plateforme web sécurisée de gestion des fournisseurs, des
appels d'offres, des candidatures et de la facturation.*

Réalisé par :

Hamza AIT OUMGHAR

Encadré par :

Pr. Oussama Lebbar

Organisme d'accueil :

Office National de l'Électricité et de l'Eau
Potable (ONEE)

Filière :

Génie Informatique

Remerciements

Au terme de ce stage de fin d'année réalisé au sein de l'Office National de l'Électricité et de l'Eau Potable (**ONEE**), je tiens à exprimer ma sincère gratitude à toutes les personnes qui, par leurs conseils, leur disponibilité et leur soutien, ont contribué à la réalisation de ce projet.

Mes remerciements les plus sincères s'adressent en premier lieu à mon encadrant pédagogique, **Pr. Oussama Lebbar**, pour la qualité de son encadrement, sa disponibilité tout au long de ce stage et la pertinence de ses remarques qui m'ont permis d'orienter correctement mes travaux. Ses conseils méthodologiques, tant sur le plan technique que sur la rédaction de ce rapport, ont été d'une aide précieuse.

Je remercie également l'équipe d'encadrement au sein de l'**ONEE** pour m'avoir accueilli au sein de leur structure, pour m'avoir fait confiance et pour m'avoir guidé sur les spécificités des marchés publics et des exigences de cybersécurité applicative formulées par la Direction Générale de la Sécurité des Systèmes d'Information (**DGSSI**). Leur expertise métier m'a permis de comprendre les contraintes réelles auxquelles une application de cette nature doit répondre.

Je tiens aussi à remercier l'ensemble du corps professoral et administratif de l'**École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la formation de qualité qu'ils m'ont dispensée durant ces années d'études, et qui m'a permis d'acquérir les compétences nécessaires à la conduite d'un tel projet, depuis l'analyse des besoins jusqu'au déploiement d'une solution opérationnelle.

Mes remerciements s'adressent aussi aux membres du jury qui ont accepté d'évaluer ce travail, pour le temps qu'ils consacreront à la lecture de ce rapport et à l'écoute de ma soutenance, ainsi que pour les remarques constructives qu'ils ne manqueront pas de m'adresser.

Enfin, je remercie toute personne ayant contribué, de quelque manière que ce soit, à la réussite de ce projet, notamment mes camarades de promotion avec qui j'ai pu échanger autour des choix techniques et des difficultés rencontrées durant le développement de l'application.

Hamza Ait Oumghar

Résumé

La gestion des relations fournisseurs constitue, pour tout organisme public gérant un volume significatif de marchés et d'achats, un enjeu opérationnel majeur. Le suivi manuel de ces relations, fréquemment réalisé à l'aide de fichiers tableurs disparates, de courriels non chiffrés et de dépôts physiques de dossiers, conduit rapidement à une perte de traçabilité, à des vérifications mal documentées et à une exposition aux risques de cybersécurité que la réglementation nationale interdit désormais.

Ce rapport présente la conception et la réalisation d'un **Portail Fournisseur** sécurisé, développé dans le cadre de mon stage de fin d'année au sein de l'Office National de l'Électricité et de l'Eau Potable (**ONEE**). L'application permet la gestion unifiée du cycle de vie des fournisseurs : auto-inscription avec validation obligatoire du numéro d'Identifiant Commun de l'Entreprise (ICE, 15 chiffres), dépôt des dossiers juridiques et administratifs, publication et gestion des appels d'offres, soumission et instruction des candidatures, et suivi de la facturation, le tout en stricte conformité avec les directives de la Direction Générale de la Sécurité des Systèmes d'Information (**DGSSI**).

Sur le plan technique, la solution repose sur les principes de la *Clean Architecture* (architecture hexagonale) implémentée avec **Spring Boot 3** et **Java 21** : un back-end exposant une API REST sécurisée par jetons JWT, et un front-end développé en HTML, CSS et JavaScript natif. La persistance des données est assurée par un système de gestion de base de données relationnelle **MySQL 8.0**, gérée par des migrations versionnées **Flyway**.

Une attention particulière a été portée à la sécurité applicative : hachage des mots de passe par **BCrypt** (facteur de coût 12), jetons de réinitialisation à usage unique hachés en SHA-256, limitation de débit sur l'authentification, en-têtes de sécurité HTTP (CSP, HSTS, X-Frame-Options), prévention de l'énumération d'utilisateurs et journalisation d'audit inaltérable.

Mots-clés : portail fournisseur, sécurité DGSSI, Spring Boot, Clean Architecture, API REST, JWT, BCrypt, MySQL, Flyway, ONEE, audit, RBAC.

Abstract

Supplier relationship management represents, for any public institution handling a significant volume of procurement contracts, a major operational challenge. Manually tracking such relationships, often through scattered spreadsheet files, unencrypted emails and physical document submissions, quickly leads to a loss of traceability, poorly documented verifications and exposure to cybersecurity risks that national regulations now prohibit.

This report presents the design and development of a secure **Supplier Portal**, built during my end-of-year internship at the National Office of Electricity and Drinking Water (**ONEE**). The application enables unified management of the supplier lifecycle: self-registration with mandatory Moroccan corporate tax identifier (ICE, 15 digits) validation, legal and administrative document uploads, tender publication and management, bid submission and review, and invoice tracking, all in strict compliance with directives issued by the General Directorate of Information Systems Security (**DGSSI**).

From a technical standpoint, the solution relies on *Clean Architecture* (hexagonal architecture) principles implemented with **Spring Boot 3** and **Java 21**: a back-end exposing a REST API secured through JWT tokens, and a front-end developed in native HTML, CSS and JavaScript. Data persistence is handled by a **MySQL 8.0** relational database management system, managed through versioned **Flyway** migrations.

Particular attention was paid to application security: **BCrypt** password hashing (cost factor 12), single-use SHA-256 hashed password reset tokens, authentication rate limiting, HTTP security headers (CSP, HSTS, X-Frame-Options), user enumeration prevention and an append-only audit trail.

Keywords: supplier portal, DGSSI compliance, Spring Boot, Clean Architecture, REST API, JWT, BCrypt, MySQL, Flyway, ONEE, audit logging, RBAC.

Table des matières

Introduction générale.....	8
Contexte général	8
Problématique.....	8
Objectifs du projet	8
Méthodologie de travail.....	9
Organisation du rapport	9
Généralités sur le projet	10
1.1 Introduction	10
1.2 Présentation de l'organisme d'accueil	10
1.2.1 Missions principales.....	10
1.2.2 Contexte numérique et enjeux de cybersécurité	10
1.3 Étude de l'existant.....	10
1.4 Expression du besoin	11
1.5 Conclusion.....	13
Conception du projet	14
2.1 Introduction	14
2.2 Architecture générale de la solution	14
2.2.1 Architecture en couches du back-end : Clean Architecture	15
2.3 Diagramme de cas d'utilisation	16
2.4 Diagramme de classes	17
2.5 Conception de la base de données	18
2.6 Conclusion.....	19
Réalisation du projet.....	20
3.1 Introduction	20
3.2 Outils et langages de programmation utilisés	20
3.3 Architecture technique détaillée	21
3.4 Sécurité de l'application.....	23
3.5 Le module de messagerie électronique	25
3.6 Interfaces de l'application	26
3.7 Conclusion.....	27
Conclusion générale et perspectives	28
Bibliographie et webographie	31
Annexes	32

Liste des figures

- Figure 2.1 : Architecture générale du Portail Fournisseur
- Figure 2.2 : Organisation du back-end selon la Clean Architecture
- Figure 2.3 : Cas d'utilisation selon les profils utilisateur
- Figure 2.4 : Vue synthétique des entités métier
- Figure 3.1 : Chaîne de sécurisation des opérations
- Figure 3.2 : Illustration de l'interface du Portail Fournisseur

Liste des tableaux

- Tableau 1.1 : Acteurs et responsabilités principales
- Tableau 1.2 : Synthèse des besoins non fonctionnels
- Tableau 2.1 : Description des classes principales
- Tableau 3.1 : Technologies back-end
- Tableau 3.2 : Technologies front-end
- Tableau 3.3 : Principaux points d'entrée de l'API REST
- Tableau 3.4 : Matrice de contrôle d'accès par rôle

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface (interface de programmation applicative)
BCrypt	Algorithme adaptatif de hachage de mots de passe (basé sur Blowfish)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete (créer, lire, mettre à jour, supprimer)
CSP	Content Security Policy
CSS	Cascading Style Sheets (feuilles de style en cascade)
DGSSI	Direction Générale de la Sécurité des Systèmes d'Information
DNSSI	Directive Nationale de la Sécurité des Systèmes d'Information
DTO	Data Transfer Object
EMSI	École Marocaine des Sciences de l'Ingénieur
HTML	HyperText Markup Language
HSTS	HTTP Strict Transport Security
HTTP	HyperText Transfer Protocol
ICE	Identifiant Commun de l'Entreprise (15 chiffres)
IHM	Interface Homme-Machine
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données
MRD	Modèle Relationnel de Données
ONEE	Office National de l'Électricité et de l'Eau Potable
ORM	Object-Relational Mapping (mapping objet-relationnel)
OWASP	Open Web Application Security Project
RBAC	Role-Based Access Control (contrôle d'accès basé sur les rôles)
REST	Representational State Transfer
SGBDR	Système de Gestion de Base de Données Relationnelle
SHA	Secure Hash Algorithm
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UML	Unified Modeling Language

Introduction générale

Le numérique occupe aujourd'hui une place centrale dans le fonctionnement de toute organisation, qu'elle soit publique ou privée. Les établissements publics marocains, en particulier les opérateurs d'importance vitale comme l'Office National de l'Électricité et de l'Eau Potable (**ONEE**), interagissent quotidiennement avec des centaines de partenaires commerciaux : fournisseurs de biens, prestataires de services et exécutants de travaux de génie civil. La gestion de ces relations, depuis l'enregistrement initial d'un fournisseur jusqu'au règlement de ses factures, constitue un flux critique que la transformation numérique ne peut plus ignorer.

Contexte général

Face à l'augmentation continue des cybermenaces, la Direction Générale de la Sécurité des Systèmes d'Information (**DGSSI**), rattachée à l'Administration de la Défense Nationale, a instauré un cadre réglementaire strict, la Directive Nationale de la Sécurité des Systèmes d'Information (**DNSSI**), visant à prémunir les systèmes d'information des organismes publics contre les fuites d'informations, l'usurpation d'identité et les accès non autorisés. Toute application web exposée à des acteurs externes doit désormais intégrer, dès sa conception, un ensemble de mesures de sécurité couvrant l'authentification, le chiffrement, la traçabilité et le cloisonnement des droits d'accès.

C'est dans ce contexte que s'inscrit mon stage de fin d'année : la conception et la réalisation d'un portail fournisseur sécurisé pour l'ONEE, en conformité avec les exigences de la DGSSI.

Problématique

Les canaux traditionnels de gestion des fournisseurs au sein de l'ONEE posent plusieurs problèmes concrets que j'ai cherché à résoudre au travers de ce projet :

- Comment garantir, à tout instant, une vision exhaustive et à jour de l'ensemble des fournisseurs référencés, de leur statut administratif et de la validité de leurs pièces justificatives ?
- Comment assurer une traçabilité irréfutable des dépôts de candidatures aux appels d'offres et des soumissions de factures, en l'absence de système centralisé ?
- Comment empêcher l'enregistrement d'entreprises fictives sans validation préalable du numéro ICE officiel, et comment vérifier systématiquement la conformité des dossiers avant toute admission au référentiel fournisseurs ?
- Comment protéger l'application contre les attaques applicatives courantes (injection, énumération d'utilisateurs, vol de session, force brute) tout en garantissant une expérience utilisateur fluide ?

Ces interrogations ont guidé l'ensemble des choix de conception et de réalisation présentés dans ce rapport.

Objectifs du projet

Le projet poursuit les objectifs suivants :

1. Mettre en place un portail d'auto-inscription sécurisée des entreprises avec contrôle systématique de l'ICE (15 chiffres obligatoires) et validation administrative préalable à l'activation du compte.
2. Permettre la gestion complète du cycle de vie des appels d'offres : création, publication, clôture et annulation, avec soumission et instruction des candidatures fournisseurs.
3. Assurer le suivi de la facturation liée aux marchés attribués, avec un circuit de validation à plusieurs étapes.

4. Implémenter l'ensemble des règles d'hygiène informatique et de sécurité DGSSI : authentification stateless par jeton JWT signé, hachage des mots de passe par BCrypt avec facteur de coût 12, jetons de réinitialisation à usage unique hachés en SHA-256, limitation de débit sur l'authentification et en-têtes de sécurité HTTP.
5. Adopter une architecture logicielle propre (*Clean Architecture* / architecture hexagonale) facilitant les tests, la maintenance et l'évolution du code.
6. Garantir la traçabilité inaltérable de toutes les opérations sensibles via un journal d'audit *append-only*.

Méthodologie de travail

Le développement du projet a suivi une démarche itérative et incrémentale, inspirée des principes agiles. Après une phase initiale d'analyse des besoins et de modélisation, j'ai avancé par cycles courts, en mettant en place progressivement les différents modules : gestion des fournisseurs, puis des documents administratifs, puis des appels d'offres et des candidatures, puis la facturation, et enfin les fonctionnalités transverses (notifications, journal d'audit, module de courrier électronique). Chaque module a été testé avant son intégration définitive, ce qui a limité les régressions au fil de l'avancement du projet.

Organisation du rapport

Ce rapport est structuré en quatre chapitres.

Le **premier chapitre** présente le cadre général du projet : la présentation de l'organisme d'accueil, la problématique détaillée, l'expression des besoins fonctionnels et non fonctionnels, ainsi qu'une brève étude de l'existant.

Le **deuxième chapitre** est consacré à la phase de conception : architecture globale de la solution, modélisation UML (diagrammes de cas d'utilisation, diagramme de classes) et conception de la base de données.

Le **troisième chapitre** détaille la phase de réalisation : les outils, langages et frameworks mobilisés, l'architecture technique mise en place, les mécanismes de sécurité DGSSI implémentés, ainsi qu'une présentation commentée des principales interfaces de l'application.

Enfin, une **conclusion générale** dresse le bilan du travail accompli, revient sur les difficultés rencontrées et propose des perspectives d'évolution.

CHAPITRE 1

Généralités sur le projet

1.1 Introduction

Avant d'entrer dans le détail de la conception et du code, il m'a paru utile de revenir sur le point de départ du projet : pourquoi ce portail fournisseur, et pour répondre à quel besoin réel. Ce premier chapitre pose donc le cadre général de mon travail. Je commence par la présentation de l'organisme d'accueil, puis un tour d'horizon de ce qui existe déjà en matière de gestion des fournisseurs, avant de détailler les besoins fonctionnels et non fonctionnels recueillis et les différents profils d'utilisateurs appelés à se servir de l'application.

1.2 Présentation de l'organisme d'accueil

L'Office National de l'Électricité et de l'Eau Potable (ONEE) est un établissement public à caractère industriel et commercial, né en 2012 de la fusion de l'Office National de l'Électricité (ONE) et de l'Office National de l'Eau Potable (ONEP). Il constitue le pilier du service public de l'énergie et de l'eau au Maroc.

1.2.1 Missions principales

L'ONEE assure trois missions fondamentales :

- **Production, transport et distribution de l'énergie électrique** sur l'ensemble du territoire national.
- **Planification, production et distribution de l'eau potable** pour les populations urbaines et rurales.
- **Gestion de l'assainissement liquide** dans les centres relevant de son périmètre d'intervention.

1.2.2 Contexte numérique et enjeux de cybersécurité

En tant qu'opérateur d'importance vitale (OIV) au sens de la réglementation marocaine, l'ONEE est soumis à des obligations renforcées en matière de sécurité des systèmes d'information. La DGSSI impose à ces organismes un ensemble de mesures couvrant l'authentification forte, le chiffrement des données sensibles, la journalisation des accès et le cloisonnement strict des droits. Toute application exposée à des acteurs externes — fournisseurs, prestataires ou partenaires — doit intégrer ces exigences dès sa phase de conception.

C'est dans ce cadre réglementaire que s'inscrit le développement du portail fournisseur objet de ce stage.

1.3 Étude de l'existant

Avant de se lancer dans la conception, j'ai pris le temps de regarder ce qui existait déjà, aussi bien les pratiques observées au sein de l'ONEE que les solutions logicielles disponibles sur le marché.

1.3.1 Pratiques existantes à l'ONEE

Au sein de l'ONEE, la gestion des relations fournisseurs reposait encore en grande partie sur des processus manuels : dépôt physique des dossiers de candidature, échanges de courriels non chiffrés pour la transmission de pièces justificatives, suivi des appels d'offres dans des fichiers tableurs partagés sur un serveur de fichiers interne. Cette approche présentait des limites bien connues des services achats : pas de traçabilité fiable des dépôts, risque d'erreurs lors de la vérification des pièces légales, difficulté à retrouver l'historique d'un fournisseur donné, et surtout aucune garantie de conformité avec les exigences de la DGSSI en matière de sécurité des échanges.

1.3.2 Solutions logicielles existantes

Il existe sur le marché des solutions commerciales de gestion de la relation fournisseur (Supplier Relationship Management ou SRM), généralement proposées en mode SaaS. Elles offrent des fonctionnalités poussées : portail d'auto-inscription, gestion documentaire, workflows de validation automatisés, intégration avec des systèmes ERP. Le revers de la médaille, c'est le coût de licence, qui reste significatif, et surtout l'hébergement des données sur des serveurs externes, ce qui pose un problème de souveraineté et de conformité réglementaire pour un organisme d'importance vitale comme l'ONEE, tenu de conserver la maîtrise de ses données sensibles.

1.3.3 Synthèse et positionnement de la solution

Cette étude de l'existant a fini de me convaincre de l'intérêt d'une solution sur mesure, légère et entièrement adaptée aux besoins exprimés : une application web autonome, sans licence tierce, déployable sur l'infrastructure interne de l'ONEE, et dont les règles métier — processus de validation, profils d'utilisateurs, exigences de sécurité DGSSI — collent au fonctionnement réel de l'organisme plutôt qu'à un référentiel générique.

1.4 Expression du besoin

1.4.1 Identification des acteurs

L'analyse des besoins m'a conduit à distinguer trois profils d'utilisateurs, chacun avec un périmètre d'action qui lui est propre au sein de l'application.

Acteur	Responsabilités principales
Administrateur	Supervision globale de l'application : gestion des comptes utilisateurs et de leurs rôles, validation ou rejet des comptes fournisseurs en attente, consultation du journal d'audit, gestion des fiches fournisseurs et supervision générale de la plateforme.
Service Achat	Gestion opérationnelle des marchés : publication des appels d'offres, instruction des candidatures fournisseurs (acceptation, rejet), validation des dossiers fournisseurs, traitement des factures, consultation et suivi des fournisseurs référencés.
Fournisseur	Utilisateur externe : auto-inscription avec numéro ICE, téléversement des pièces légales (registre de commerce, attestation fiscale, CNSS, RIB), consultation et réponse aux appels d'offres publiés, soumission et suivi des factures, consultation de sa propre fiche fournisseur.

Tableau 1.1 : Acteurs du système et leurs responsabilités principales

Cette distinction stricte répond à un principe fondamental de sécurité applicative, le *principe du moindre privilège* : chaque utilisateur ne doit disposer que des droits strictement nécessaires à l'accomplissement de ses tâches. Un fournisseur, par exemple, ne peut en aucun cas consulter les fiches d'autres fournisseurs, ni accéder au journal d'audit : il n'a accès qu'à son propre périmètre.

1.4.2 Besoins fonctionnels

J'ai regroupé les besoins fonctionnels exprimés pour le projet par module.

Module Authentification et gestion des comptes

- Authentification sécurisée par identifiant (nom d'utilisateur) et mot de passe.
- Auto-inscription des fournisseurs avec création d'un compte en statut « en attente de validation », nécessitant une approbation administrative avant activation.
- Génération et envoi d'un mot de passe temporaire par courriel lors de la validation du compte fournisseur.
- Mécanisme de réinitialisation de mot de passe par courrier électronique, avec lien à usage unique et à durée de validité limitée.
- Obligation de changement de mot de passe lors de la première connexion avec un mot de passe temporaire.

Module Gestion des fournisseurs

- Enregistrement du profil fournisseur : raison sociale, numéro ICE (15 chiffres obligatoires), registre de commerce, identifiant fiscal, adresse, personne de contact, catégorie d'activité.
- Cycle de vie du statut fournisseur : DRAFT → PENDING_VALIDATION → VALIDATED ou REJECTED, avec possibilité de suspension (SUSPENDED) et de réintégration.
- Recherche et filtrage des fournisseurs par statut, catégorie et mot-clé, avec pagination côté serveur.

Module Documents administratifs

- Téléversement des pièces justificatives : extrait du registre de commerce, attestation de conformité fiscale, attestation CNSS, certificat ICE, certification ISO, relevé d'identité bancaire.
- Cycle de validation de chaque document : PENDING_REVIEW → APPROVED ou REJECTED, avec possibilité de marquer un document comme expiré (EXPIRED).
- Suivi des dates d'expiration des pièces et alerte en cas de dépassement.

Module Appels d'offres et candidatures

- Création, modification, publication, clôture et annulation des appels d'offres par le service achat.
- Consultation des appels d'offres publiés par les fournisseurs validés.
- Soumission d'une candidature par le fournisseur : proposition technique et montant proposé.
- Instruction de la candidature par le service achat : acceptation ou rejet avec commentaire motivé.
- Contrainte d'unicité : un fournisseur ne peut soumettre qu'une seule candidature par appel d'offres.

Module Facturation

- Émission d'une facture par le fournisseur, liée à une candidature acceptée.
- Cycle de validation : EN_ATTENTE → VALIDEE → PAYEE, ou EN_ATTENTE → REJETEE.
- Suivi des dates d'échéance et des montants.

Module Audit et traçabilité

- Enregistrement systématique de toute action sensible dans un journal d'audit inaltérable : connexion réussie ou échouée, réinitialisation de mot de passe, enregistrement d'un fournisseur, validation, rejet, téléversement de document, etc.
- Chaque entrée du journal consigne l'identité de l'acteur, le type d'action, l'entité cible, l'adresse IP, le résultat et un commentaire détaillé.

1.4.3 Besoins non fonctionnels

Indépendamment des fonctionnalités métier, plusieurs exigences non fonctionnelles ont guidé mes choix d'architecture et de réalisation :

Critère	Exigence
Sécurité	Authentification par jeton signé (JWT), mots de passe hachés (BCrypt, facteur 12), jetons de réinitialisation à usage unique (SHA-256), limitation de débit sur l'authentification, en-têtes de sécurité HTTP (CSP, HSTS, X-Frame-Options), secrets externalisés.
Conformité DGSSI	Respect de la Directive Nationale de la Sécurité des Systèmes d'Information : non-énumération des comptes, journalisation d'audit inaltérable, séparation des privilèges.
Performance	Temps de réponse perçu inférieur à la seconde pour les opérations de consultation courantes.
Ergonomie	Interface claire, cohérente, sans surcharge visuelle, adaptée à un usage professionnel quotidien, y compris par des fournisseurs non spécialistes en informatique.
Portabilité	Application accessible depuis un simple navigateur web, sans installation cliente, backend packagé sous forme d'archive Java exécutable de manière autonome.

Critère	Exigence
Maintenabilité	Code structuré selon les principes de la Clean Architecture, séparation claire des responsabilités entre couches domaine, application et infrastructure.
Extensibilité	Architecture permettant l'ajout aisé de nouveaux modules sans remise en cause de l'existant.

Tableau 1.2 : Synthèse des besoins non fonctionnels

1.5 Conclusion

Nous disposons à présent d'une vision claire du projet : son contexte, la problématique qu'il cherche à résoudre, ce qui existe déjà, et l'ensemble des besoins fonctionnels et non fonctionnels recueillis. C'est sur cette base que repose tout le travail de conception qui suit, où ces besoins se traduisent concrètement en architecture logicielle et en modélisation UML.

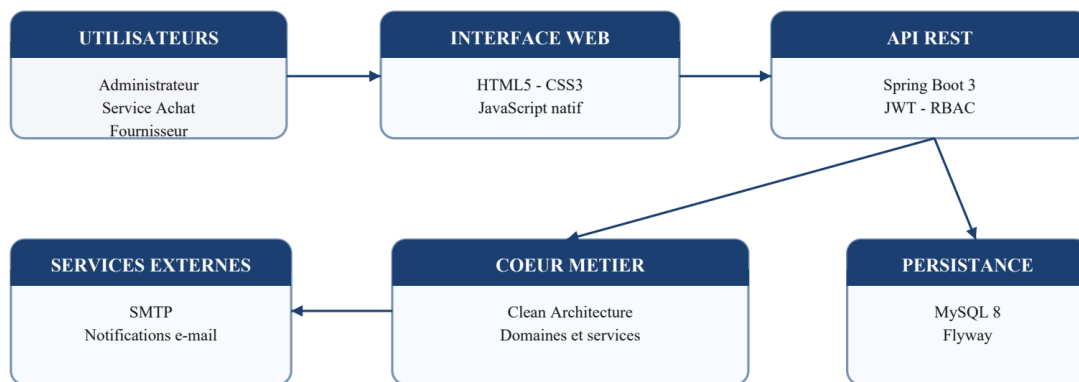
CHAPITRE 2

Conception du projet

2.1 Introduction

Le chapitre précédent a posé le contexte et les besoins du projet ; voyons à présent comment ces besoins se traduisent en choix de conception. Ce chapitre couvre l'architecture logicielle retenue, la modélisation des cas d'utilisation pour chaque profil d'utilisateur, le diagramme de classes qui décrit la structure statique de l'application, et la conception de la base de données.

2.2 Architecture générale de la solution



Architecture générale du Portail Fournisseur

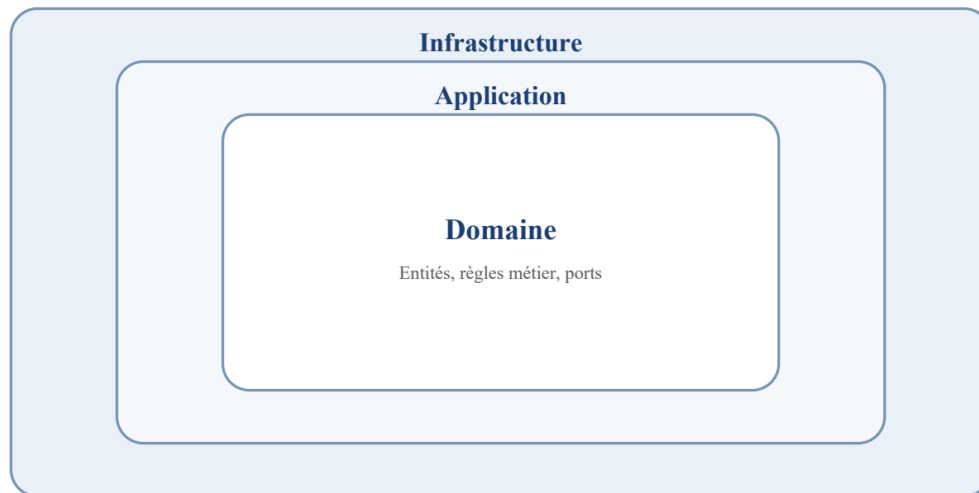
Figure 2.1 : Architecture générale du Portail Fournisseur.

L'application repose sur une architecture client-serveur à trois niveaux (*three-tier architecture*). C'est un schéma classique dans le développement web moderne, qui a l'avantage de bien séparer les responsabilités :

- une **couche de présentation** (front-end), exécutée dans le navigateur de l'utilisateur, responsable de l'affichage des données et de la capture des interactions ;
- une **couche métier** (back-end), responsable de l'application des règles de gestion, du contrôle d'accès et de l'orchestration des traitements ;
- une **couche de persistance**, assurée par un système de gestion de base de données relationnelle, responsable du stockage durable des données.

Le front-end et le back-end ne communiquent que par une API REST exposée par le serveur, avec des échanges de données sérialisés en JSON. Cette séparation stricte a plusieurs avantages : on peut faire évoluer l'interface et la logique métier indépendamment l'une de l'autre, on garde la possibilité de réutiliser la même API pour un futur client (une application mobile, par exemple), et les responsabilités de sécurité restent sans ambiguïté : seul le serveur est habilité à valider les droits d'accès.

2.2.1 Architecture en couches du back-end : Clean Architecture



Les dépendances pointent vers le domaine métier, indépendant des technologies.

Figure 2.2 : Organisation du back-end selon la Clean Architecture.

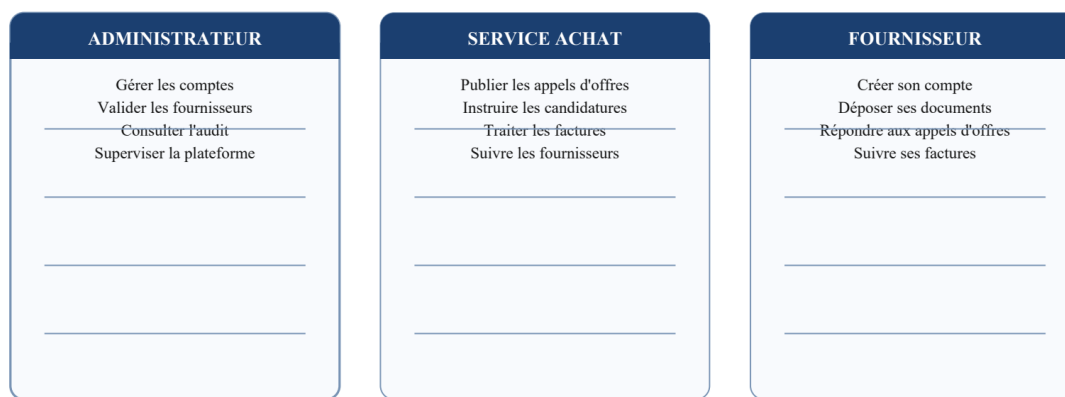
Contrairement à une architecture en couches classique (contrôleur → service → repository), le back-end de ce projet suit les principes de la **Clean Architecture** (ou *architecture hexagonale*), telle que formalisée par Robert C. Martin. L'idée fondatrice est que le **domaine métier** ne doit dépendre d'aucun framework, d'aucune base de données, d'aucun mécanisme de transport. Les dépendances pointent toujours vers l'intérieur.

Concrètement, le code est organisé en quatre couches concentriques :

- **Couche Domaine ('domain')** : le cœur métier de l'application. Contient les entités (User, Supplier, AppelOffre, Demande, Facture, SupplierDocument, AuditLogEntry), les objets de valeur (IceNumber, Email), les énumérations de statut et les contrats d'interfaces de persistance (UserRepository, SupplierRepository, etc.). Cette couche n'a **aucune dépendance** vers Spring, JPA ou tout autre framework.
- **Couche Application ('application')** : contient les services d'orchestration métier (AuthenticationService, SupplierService, AppelOffreService, DocumentService, etc.), les commandes (RegisterCommand, LoginCommand, CreateSupplierCommand) et les objets de résultat. Elle coordonne les appels aux ports de la couche domaine sans connaître les détails d'implémentation.
- **Couche Infrastructure ('infrastructure')** : fournit les implémentations concrètes des contrats définis par le domaine. C'est ici que vivent les adaptateurs JPA (SupplierRepositoryImpl, UserRepositoryImpl), les mappers entre entités de domaine et entités JPA, la configuration Spring Security, le fournisseur de jetons JWT (JwtTokenProvider), le filtre d'authentification (JwtAuthenticationFilter), le service d'envoi de courriels (EmailService) et le limiteur de débit (AuthenticationRateLimiter).
- **Couche Web ('infrastructure.web')** : les contrôleurs REST qui exposent l'API, les objets de transfert de données (DTO) pour les requêtes et les réponses, et les gestionnaires d'erreurs.

Cette organisation garantit que le domaine métier reste pur et testable indépendamment : si demain on décide de remplacer MySQL par PostgreSQL, ou Spring Security par un autre framework, seule la couche infrastructure est impactée ; le code métier reste inchangé.

2.3 Diagramme de cas d'utilisation



Cas d'utilisation principaux par profil utilisateur

Figure 2.3 : Principaux cas d'utilisation selon les trois profils du système.

Le diagramme de cas d'utilisation donne une vue synthétique des interactions entre les acteurs du système et des fonctionnalités auxquelles chacun a accès.

2.3.1 Cas d'utilisation de l'administrateur

L'administrateur dispose du périmètre fonctionnel le plus large dans l'application :

- Gérer les comptes utilisateurs (création, modification, activation/désactivation, attribution des rôles).
- Valider ou rejeter les comptes fournisseurs en attente de validation.
- Créer manuellement des fiches fournisseurs.
- Consulter et gérer l'ensemble des fournisseurs, documents, appels d'offres, candidatures et factures.
- Consulter le journal d'audit complet.
- Consulter le tableau de bord global avec les indicateurs clés de la plateforme.

2.3.2 Cas d'utilisation du service achat

Le service achat assure la gestion opérationnelle des marchés :

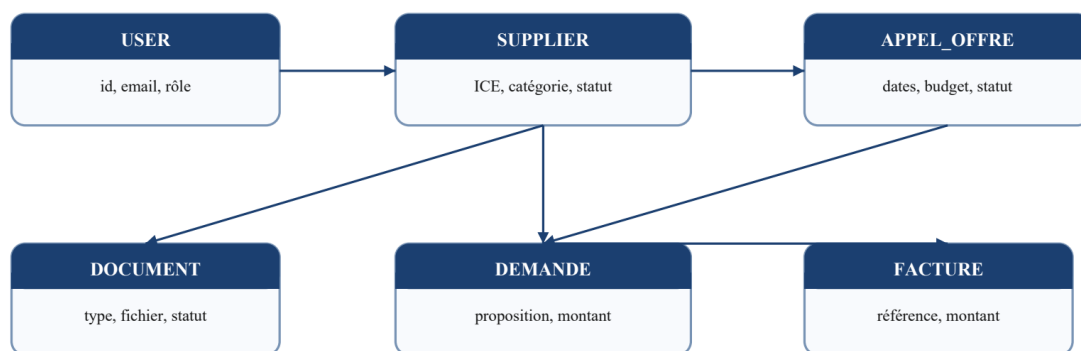
- Créer, modifier, publier, clôturer et annuler des appels d'offres.
- Instruire les candidatures fournisseurs (acceptation ou rejet avec commentaire motivé).
- Valider ou rejeter les dossiers fournisseurs et leurs pièces justificatives.
- Valider ou rejeter les factures soumises par les fournisseurs.
- Suspendre ou réintégrer un fournisseur.

2.3.3 Cas d'utilisation du fournisseur

Le fournisseur est un utilisateur externe au périmètre strictement limité :

- S'inscrire avec un numéro ICE valide (15 chiffres) et attendre la validation administrative.
- Consulter et modifier sa fiche fournisseur (uniquement en statut brouillon).
- Soumettre sa fiche pour validation.
- Téléverser ses pièces justificatives (registre de commerce, attestation fiscale, CNSS, RIB, certificat ICE, certification ISO).
- Consulter les appels d'offres publiés et y soumettre une candidature.
- Émettre des factures liées à des candidatures acceptées.
- Consulter l'état de ses propres candidatures et factures.

2.4 Diagramme de classes



Vue simplifiée des principales entités du modèle relationnel

Figure 2.4 : Vue synthétique des entités métier principales.

Le diagramme de classes est le pivot de la modélisation orientée objet de l'application : il décrit la structure statique du domaine métier, les attributs de chaque entité et les relations qui les unissent. C'est directement sur cette base que j'ai défini les entités de domaine et les tables de la base de données.

2.4.1 Description des classes principales

Classe	Description
User	Représente un utilisateur de l'application. Possède un nom d'utilisateur unique, une adresse électronique (objet de valeur <code>Email</code>), un mot de passe haché, un rôle (<code>ADMIN</code> , <code>SERVICE_ACHAT</code> , <code>SUPPLIER</code>), un statut (<code>ACTIVE</code> , <code>LOCKED</code> , <code>DISABLED</code>), un compteur d'échecs de connexion et un indicateur de changement de mot de passe obligatoire. Encapsule la règle de verrouillage automatique après 5 tentatives échouées.
Supplier	Représente le profil d'un fournisseur enregistré. Associé à un <code>IceNumber</code> (15 chiffres obligatoires, objet de valeur), une catégorie d'activité parmi 21 catégories prédéfinies (travaux, fournitures, services) et un statut de cycle de vie (<code>DRAFT</code> , <code>PENDING_VALIDATION</code> , <code>VALIDATED</code> , <code>REJECTED</code> , <code>SUSPENDED</code>). Encapsule les règles de transitions d'état autorisées.
SupplierDocument	Représente une pièce justificative déposée par un fournisseur : type de document (7 types prédéfinis), référence de fichier, date d'expiration, statut de validation (<code>PENDING_REVIEW</code> , <code>APPROVED</code> , <code>REJECTED</code> , <code>EXPIRED</code>). Encapsule les règles de validation avec identification du réviseur.
AppelOffre	Représente un appel d'offres publié par le service achat : titre, description, catégorie, budget estimé, dates de publication et de clôture, statut (<code>BROUILLON</code> , <code>PUBLIE</code> , <code>CLOS</code> , <code>ANNULE</code>). Encapsule les règles de transitions d'état et la vérification d'ouverture.
Demande	Représente la candidature d'un fournisseur à un appel d'offres : proposition technique, montant proposé, statut (<code>SOUmise</code> , <code>ACCEPTEE</code> , <code>REJETEE</code>), commentaire du service achat.
Facture	Représente une facture émise par un fournisseur : référence, montant, dates d'émission et d'échéance, statut (<code>EN_ATTENTE</code> , <code>VALIDEE</code> , <code>PAYEE</code> , <code>REJETEE</code>).
AuditLogEntry	Entité immuable enregistrant chaque action sensible : horodatage, identité de l'acteur, type d'action (24 types prédéfinis), entité cible, adresse IP, résultat et commentaire détaillé.
IceNumber	Objet de valeur encapsulant le numéro ICE : validation par expression régulière (<code>^\d{15}\$</code>), constructeur privé et méthode de fabrique statique <code>of(String)</code> .
Email	Objet de valeur encapsulant l'adresse électronique : validation par expression régulière, immuable.

Tableau 2.1 : Description des principales classes du domaine

2.4.2 Relations principales

Les relations entre classes traduisent directement les règles de gestion du domaine :

- Un `User` possède exactement un `Role` (attribut énuméré) ;
- Un `Supplier` est rattaché à un `User` qui l'a enregistré (clé étrangère `registered_by_user_id`) ;
- Un `Supplier` peut déposer plusieurs `SupplierDocument` au fil du temps ;
- Un `AppelOffre` est publié par un `User` du rôle `SERVICE_ACHAT` ;
- Un `AppelOffre` peut recevoir plusieurs `Demande`, mais un même fournisseur ne peut soumettre qu'une seule candidature par appel d'offres (contrainte d'unicité en base) ;
- Une `Demande` acceptée peut donner lieu à une ou plusieurs `Facture` ;
- Un `User` peut être à l'origine de plusieurs jetons de réinitialisation au fil du temps.

2.5 Conception de la base de données

La base de données relationnelle assure la persistance de l'ensemble des entités décrites plus haut. J'ai suivi les principes de normalisation classiques pour limiter la redondance des données et garantir leur cohérence.

2.5.1 Modèle conceptuel de données

Le modèle conceptuel de données (MCD) traduit les entités du domaine et leurs associations sous une forme indépendante de toute technologie, avec les cardinalités correspondantes. Il se concentre sur le cœur métier relié par des clés étrangères : utilisateurs, fournisseurs, documents, appels d'offres, candidatures, factures et audit.

2.5.2 Passage au modèle relationnel

Le passage du modèle conceptuel au modèle relationnel a donné les tables suivantes, gérées par six migrations Flyway versionnées (V1 à V6) :

- ``users`` (id, username, email, password_hash, role, status, failed_login_attempts, must_change_password, last_login_at, created_at, updated_at)
- ``suppliers`` (id, company_name, ice, commercial_register, tax_identifier, address, contact_person, contact_email, contact_phone, category, status, registered_by_user_id, created_at, updated_at)
- ``supplier_documents`` (id, supplier_id, document_type, file_reference, uploaded_at, expiry_date, status, reviewed_by_user_id, review_comment, reviewed_at)
- ``supplier_evaluations`` (id, supplier_id, evaluator_user_id, evaluation_date, overall_score, comment, created_at)
- ``evaluation_criterion_scores`` (id, evaluation_id, criterion, score)
- ``appel_offres`` (id, titre, description, categorie, budget_estime, date_publication, date_cloture, status, published_by_user_id, created_at, updated_at)
- ``demandes`` (id, appel_offre_id, fournisseur_user_id, proposition_technique, montant_propose, date_submission, status, commentaire_service_achat, created_at, updated_at)
- ``factures`` (id, demande_id, fournisseur_user_id, reference_facture, montant, date_emission, date_echeance, status, commentaire, created_at, updated_at)
- ``audit_log`` (id, event_timestamp, actor_user_id, action_type, target_entity_type, target_entity_id, ip_address, outcome, detail)
- ``password_reset_tokens`` (id, user_id, token_hash, expiry_date, used, created_at)
- ``notifications`` (id, recipient_user_id, type, message, is_read, created_at)

Chaque table possède sa clé primaire auto-incrémentée, et les associations sont matérialisées par des clés étrangères assorties de contraintes d'intégrité référentielle. Des index ont été ajoutés sur les colonnes

fréquemment filtrées (`status`, `category`, `date_cloture`) pour garantir des temps de réponse acceptables à mesure que le volume de données croît.

En poussant la normalisation jusqu'à la troisième forme normale, on évite toute duplication d'information : les catégories de fournisseurs sont stockées sous forme d'énumération plutôt que dans une table de référence séparée, un choix justifié par leur caractère stable et prédéfini (21 catégories correspondant aux métiers de l'ONEE Branche Eau).

2.6 Conclusion

Les fondations conceptuelles du portail fournisseur sont maintenant posées : une architecture client-serveur à trois niveaux, un back-end structuré selon les principes de la Clean Architecture, un diagramme de classes cohérent, des cas d'utilisation qui reflètent fidèlement les besoins de chaque profil, et une base de données relationnelle normalisée gérée par des migrations versionnées. Reste à voir comment tout cela prend forme dans le code : c'est l'objet du chapitre suivant.

CHAPITRE 3

Réalisation du projet

3.1 Introduction

Le cadre général et la conception ayant été posés dans les deux premiers chapitres, place maintenant à la réalisation concrète du projet. Ce chapitre couvre les outils et langages utilisés, l'architecture technique effectivement mise en place, les mécanismes de sécurité DGSSI implémentés dans le code, le module de messagerie électronique, et une présentation commentée des principales interfaces exploitables par chaque profil d'utilisateur.

3.2 Outils et langages de programmation utilisés

Le choix des technologies a été guidé par quelques critères simples : la robustesse et la maturité des outils, la richesse de leur documentation et de leur écosystème, et leur adéquation avec les exigences de sécurité et de maintenabilité exprimées au chapitre 1.

3.2.1 Back-end

Technologie	Rôle dans le projet
Java 21 (JDK LTS)	Langage de programmation principal du back-end, choisi pour sa robustesse, son typage statique et la maturité de son écosystème.
Spring Boot 3	Framework de développement d'applications Java, utilisé pour la mise en place de l'API REST, l'injection de dépendances et la configuration simplifiée du serveur applicatif embarqué.
Spring Security 6	Module de sécurisation de l'application : gestion de l'authentification, du chiffrement des mots de passe et du contrôle d'accès par annotation sur chaque point d'entrée de l'API.
Spring Data JPA / Hibernate	Couche de persistance objet-relationnelle, utilisée pour la définition des entités et la génération automatisée des requêtes d'accès aux données.
JSON Web Token (JWT)	Mécanisme de jeton signé utilisé pour matérialiser l'authentification d'un utilisateur après connexion, sans nécessiter de session stockée côté serveur.
Flyway	Outil de gestion des migrations SQL versionnées (V1 à V6), garantissant la reproductibilité et la traçabilité de l'évolution du schéma de base de données.
Maven	Outil de gestion des dépendances et de construction (compilation, packaging) du projet back-end.
MySQL 8.0	Système de gestion de base de données relationnelle utilisé pour la persistance de l'ensemble des données de l'application.
Lombok	Bibliothèque de génération de code répétitif (getters, setters, constructeurs, builders) réduisant le boilerplate Java.

Tableau 3.1 : Technologies utilisées pour le développement du back-end

3.2.2 Front-end

Technologie	Rôle dans le projet
HTML5	Structuration du contenu des interfaces de l'application.
CSS3	Mise en forme visuelle de l'application : système de variables de couleurs, mise en page responsive, transitions sobres.

Technologie	Rôle dans le projet
JavaScript (natif)	Logique d'interaction côté client : appels à l'API REST, mise à jour dynamique du contenu des pages, gestion des formulaires, sans recours à un framework front-end tiers.

Tableau 3.2 : Technologies utilisées pour le développement du front-end

J'ai délibérément choisi de ne pas utiliser de framework front-end comme React ou Angular : la taille du projet ne le justifiait pas vraiment, et je voulais surtout garder la maîtrise de chaque ligne de code produite, dans un esprit pédagogique propre à un stage de fin d'année. Cette approche, dite « vanilla JavaScript », demande malgré tout une organisation rigoureuse : un état applicatif géré à un seul endroit, des fonctions de rendu dédiées à chaque section de l'interface, et une seule fonction utilitaire pour tous les appels réseau vers l'API.

3.2.3 Outils de développement

- **IntelliJ IDEA** : environnement de développement intégré utilisé pour le développement du back-end Java.
- **Git** : système de gestion de versions utilisé pour le suivi de l'historique du code source.
- **Postman** : outil utilisé pour tester manuellement chaque point d'entrée de l'API REST durant la phase de développement, avant intégration côté front-end.
- **Gmail SMTP** : service utilisé pour l'envoi effectif des courriels transactionnels (mot de passe temporaire, réinitialisation de mot de passe), via le protocole SMTP avec authentification TLS.

3.3 Architecture technique détaillée

3.3.1 Organisation du code back-end

Le code source du back-end est organisé en paquetages reflétant directement l'architecture hexagonale présentée au chapitre 2 :

- `domain/` : entités métier pures, objets de valeur, interfaces de ports (repositories), énumérations et exceptions de domaine. Aucune dépendance vers Spring ou JPA.
- `application/` : services d'orchestration (`AuthenticationService`, `SupplierService`, `AppelOffreService`, etc.), commandes et résultats.
- `infrastructure/persistence/` : adaptateurs JPA implémentant les ports du domaine, entités JPA annotées, mappers bidirectionnels.
- `infrastructure/security/` : configuration Spring Security, filtre JWT, fournisseur de jetons, encodeur BCrypt, limiteur de débit.
- `infrastructure/email/` : service d'envoi de courriels via Jakarta Mail.
- `infrastructure/web/controller/` : contrôleurs REST.
- `infrastructure/web/dto/` : objets de transfert de données (requêtes et réponses).

Utiliser systématiquement des objets de transfert de données (DTO) plutôt que les entités JPA dans les réponses de l'API est un choix de conception que j'ai jugé important : cela évite d'exposer par erreur un champ sensible (le mot de passe haché d'un utilisateur, par exemple) et ça découple la structure interne de la base de données de ce qui est réellement exposé au client.

3.3.2 API REST

Toutes les fonctionnalités de l'application sont exposées sous forme de ressources REST, organisées par domaine métier. Chaque ressource expose les opérations GET, POST, PUT et DELETE pertinentes, conformément aux conventions REST.

Méthode	Ressource	Description
---------	-----------	-------------

Méthode	Ressource	Description
POST	/api/v1/auth/register	Auto-inscription d'un fournisseur
POST	/api/v1/auth/login	Authentification et génération du jeton JWT
POST	/api/v1/auth/forgot-password	Demande de réinitialisation de mot de passe
POST	/api/v1/auth/reset-password	Réinitialisation effective avec jeton à usage unique
POST	/api/v1/auth/change-password	Changement de mot de passe (première connexion)
GET	/api/v1/suppliers	Liste des fournisseurs (filtrage, pagination)
POST	/api/v1/suppliers/{id}/submit	Soumission du dossier fournisseur
POST	/api/v1/suppliers/{id}/validate	Validation du dossier fournisseur
POST	/api/v1/appels-offres	Création d'un appel d'offres
PUT	/api/v1/appels-offres/{id}/publier	Publication d'un appel d'offres
GET	/api/v1/appels-offres/publies	Appels d'offres publiés (accès public)
POST	/api/v1/demandes	Soumission d'une candidature
POST	/api/v1/factures	Émission d'une facture
GET	/api/v1/audit	Consultation du journal d'audit

Tableau 3.3 : Extrait des principaux points d'entrée de l'API REST

3.3.3 Extrait de code : contrôleur REST d'authentification

Le contrôleur suivant illustre la structure type d'un point d'entrée de l'API, ici pour l'authentification et l'inscription des fournisseurs, avec la mise en œuvre de la limitation de débit et du principe de non-énumération.

```

@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthenticationService authService;
    private final PasswordResetService passwordResetService;
    private final AuthenticationRateLimiter rateLimiter;

    @PostMapping("/register")
    public ResponseEntity<> register(
        @Valid @RequestBody RegisterRequest request,
        HttpServletRequest httpRequest) {
        enforceRateLimit(httpRequest, request.getUsername());
        RegisterCommand cmd = new RegisterCommand(
            request.getUsername(), request.getEmail(),
            request.getCompanyName(), request.getIce(),
            request.getCategory(), "SUPPLIER");
        AuthenticationResult result = authService.register(cmd);
        if (result.isPendingValidation()) {
            return ResponseEntity.status(HttpStatus.ACCEPTED)
                .body(Map.of("message",
                    "Votre compte a été créé. Un mot de passe "
                    + "temporaire vous sera envoyé par email une "
                    + "fois votre compte validé."));
        }
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(toResponse(result));
    }

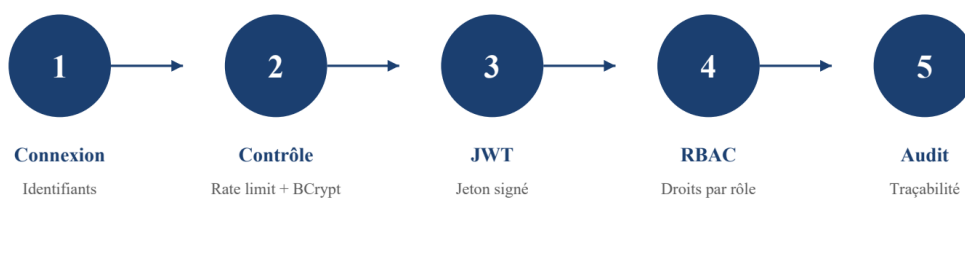
    @PostMapping("/forgot-password")
    public ResponseEntity<> forgotPassword(
        @Valid @RequestBody ForgotPasswordRequest request) {
        // Toujours 200 : ne pas révéler si l'email existe
        passwordResetService.requestPasswordReset(
            request.getEmail());
        return ResponseEntity.ok(Map.of("message",
            "Si cette adresse est associée à un compte, "
            + "un lien de réinitialisation vous a été envoyé."));
    }
}

```

Cet extrait illustre deux points essentiels de la démarche de sécurisation. D'une part, la limitation de débit (`enforceRateLimit`) protège le point d'entrée d'inscription contre les attaques par force brute. D'autre part, le

point d'entrée de réinitialisation de mot de passe retourne toujours le même message, qu'il existe ou non un compte associé à l'adresse fournie, conformément au principe de non-énumération (DGSSI).

3.4 Sécurité de l'application



Chaîne de protection appliquée aux opérations sensibles

Figure 3.1 : Chaîne de sécurisation des opérations d'authentification.

La sécurité n'a pas été traitée comme un module à part, ajouté à la fin du projet : elle a été une préoccupation présente du début à la fin du développement. Plusieurs mécanismes complémentaires ont été mis en place, en conformité avec les exigences de la DGSSI.

3.4.1 Authentification par jeton JWT

Lors d'une connexion réussie, le serveur génère un jeton JWT signé contenant l'identifiant de l'utilisateur, son rôle et sa date d'expiration. Le jeton est signé à l'aide de l'algorithme HMAC-SHA, avec une clé secrète d'au moins 32 caractères, externalisée dans la configuration. Le client transmet ensuite ce jeton à chaque requête, dans l'en-tête HTTP `Authorization`, ce qui permet au serveur de vérifier l'identité de l'appelant sans maintenir d'état de session. Cette approche, dite *stateless*, facilite la scalabilité horizontale de l'application.

Le filtre `JwtAuthenticationFilter`, placé en amont de la chaîne de filtres Spring Security, extrait le jeton de l'en-tête, le valide (signature, expiration), puis reconstitue le contexte de sécurité de l'utilisateur pour la durée de la requête.

3.4.2 Hachage des mots de passe

Aucun mot de passe n'est jamais stocké en clair dans la base de données. Chaque mot de passe est haché à l'aide de l'algorithme **BCrypt**, configuré avec un facteur de coût de 12, qui intègre un sel aléatoire propre à chaque utilisateur et rend toute attaque par force brute particulièrement coûteuse en temps de calcul. L'implémentation est fournie par l'adaptateur `BcryptPasswordEncoderAdapter`, qui encapsule le `BCryptPasswordEncoder` de Spring Security et implémente le port `PasswordEncoderPort` défini dans la couche application.

```

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
  
```

3.4.3 Contrôle d'accès par rôle (RBAC)

Chaque point d'entrée sensible de l'API est protégé par une annotation `@PreAuthorize`, évaluée côté serveur avant l'exécution de la méthode. Ce contrôle ne se limite pas à masquer ou afficher des éléments d'interface côté client : il est systématiquement révérifié côté serveur, en suivant le principe selon lequel « le client n'est jamais digne de confiance ».

Ressource	Admin	Service Achat	Fournisseur
Fournisseurs (lecture)	Tous	Tous	Le sien

Ressource	Admin	Service Achat	Fournisseur
Fournisseurs (validation)	Oui	Oui	Non
Documents (lecture)	Tous	Tous	Les siens
Documents (approbation)	Oui	Oui	Non
Appels d'offres (écriture)	Non	Oui	Non
Appels d'offres (lecture)	Oui	Oui	Publiés
Candidatures (soumission)	Non	Non	Oui
Candidatures (instruction)	Non	Oui	Non
Factures (émission)	Non	Non	Oui
Factures (validation)	Oui	Oui	Non
Utilisateurs (gestion)	Oui	Non	Non
Journal d'audit	Oui	Non	Non

Tableau 3.4 : Synthèse des droits d'accès par rôle et par ressource

3.4.4 Vérification de la propriété des données

Un point a demandé une attention particulière : il ne suffit pas qu'une ressource soit correctement filtrée par rôle si un fournisseur peut accéder aux données d'un autre fournisseur en manipulant l'identifiant dans l'URL. Le contrôleur vérifie donc systématiquement la propriété des données : avant de retourner la fiche d'un fournisseur, le serveur s'assure que l'utilisateur connecté est bien le propriétaire de cette fiche, ou bien un administrateur ou un agent du service achat.

```
private void checkOwnershipIfSupplier(Long supplierId,
                                     Principal principal) {
    User current = currentUserResolver.resolve(principal);
    if (current.getRole() == Role.SUPPLIER
        && !supplierService.isOwnedByUser(supplierId,
                                           current.getId())) {
        throw new AccessDeniedException(
            "Cette fiche fournisseur ne vous appartient pas");
    }
}
```

3.4.5 Réinitialisation sécurisée du mot de passe

Le mécanisme de réinitialisation de mot de passe a reçu une attention particulière : c'est historiquement l'une des surfaces d'attaque les plus exploitées dans les applications web. Plusieurs garanties ont été mises en place :

- le jeton de réinitialisation transmis par courriel est généré aléatoirement et n'est **jamais stocké en clair** en base de données : seule son empreinte (hachage SHA-256) y est conservée, de sorte qu'une éventuelle fuite de la base de données ne permettrait pas de reconstituer le jeton valide ;
- le jeton possède une **durée de validité limitée** (30 minutes), au-delà de laquelle il est automatiquement rejeté ;
- le jeton est à **usage unique** : toute tentative de réutilisation après une première utilisation réussie est rejetée ;
- la réponse retournée par l'API **ne distingue jamais** le cas d'une adresse électronique inexistante de celui d'un envoi réussi, afin de ne pas permettre à un attaquant de déduire la liste des comptes existants (*principe de non-énumération*).

3.4.6 Limitation de débit sur l'authentification

Le composant `AuthenticationRateLimiter` limite le nombre de tentatives d'authentification par combinaison adresse IP / nom d'utilisateur. Après dépassement du seuil, toute nouvelle tentative est rejetée avec un message explicite (« Trop de tentatives. Réessayez dans une minute. »), sans divulguer si le compte existe ou non.

3.4.7 En-têtes de sécurité HTTP

La configuration Spring Security injecte systématiquement les en-têtes de sécurité HTTP recommandés par la DGSSI et l'OWASP :

- **Content-Security-Policy (CSP)** : restriction des sources de contenu autorisées (`default-src 'self', script-src 'self', frame-ancestors 'none', etc.`).
- **X-Content-Type-Options** : prévention du MIME sniffing (`nosniff`).
- **X-Frame-Options** : interdiction de l'intégration dans un iframe (`DENY`).
- **Referrer-Policy** : limitation de la fuite d'informations dans le Referer (`strict-origin-when-cross-origin`).
- **Strict-Transport-Security (HSTS)** : forçage de l'usage de HTTPS, avec une durée de validité d'un an et inclusion des sous-domaines.

3.4.8 Externalisation des secrets de configuration

Toutes les informations sensibles de configuration (identifiants SMTP, secret de signature des jetons JWT) sont externalisées dans le fichier de configuration `application-dev.yml` et jamais codées en dur dans le code source. Ainsi, aucun secret ne se retrouve dans l'historique du gestionnaire de versions, dans les journaux applicatifs, ni dans les messages d'erreur retournés au client.

3.5 Le module de messagerie électronique

3.5.1 Service d'envoi centralisé

Tous les envois de courriels de l'application passent par le service `EmailService`, qui regroupe la logique d'envoi et évite toute duplication de code entre les différents déclencheurs. Ce service gère deux types de courriels :

- **Mot de passe temporaire** : envoyé au fournisseur lors de la validation de son compte par l'administrateur, contenant son nom d'utilisateur et un mot de passe temporaire qu'il devra changer lors de sa première connexion.
- **Lien de réinitialisation** : envoyé en réponse à une demande de réinitialisation de mot de passe, contenant un lien sécurisé à usage unique avec une durée de validité limitée.

L'envoi est effectué de manière **asynchrone** (annotation `@Async`) afin de ne jamais bloquer la requête HTTP de l'utilisateur. Si la configuration SMTP n'est pas renseignée, le service bascule automatiquement en mode simulation, consignait le contenu du courriel dans les journaux applicatifs sans erreur, ce qui facilite le développement et les tests.

3.5.2 Contenu des courriels

Les courriels sont générés en HTML directement dans le service, avec un style sobre et professionnel : en-tête au dégradé orange, contenu structuré, bloc d'information mis en valeur et bouton d'action. Ce choix de génération directe en code, plutôt que par un moteur de templates externe, limite les dépendances tout en garantissant un contrôle total sur le rendu.

3.6 Interfaces de l'application

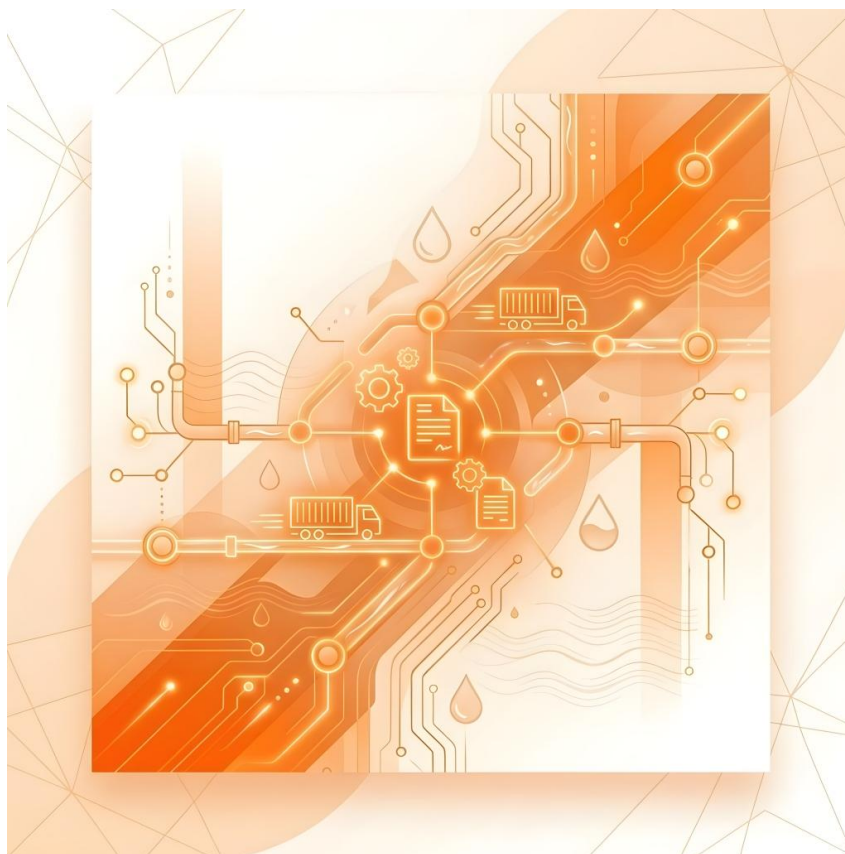


Figure 3.2 : Illustration de l'interface du Portail Fournisseur.

Cette section décrit, écran par écran, les principales interfaces de l'application : leurs fonctionnalités et leur organisation visuelle. Le nombre et la nature des interfaces présentées reflètent l'ensemble de ce qui est exploitable par les différents profils d'utilisateurs du système.

3.6.1 Interface de connexion

L'interface de connexion constitue le point d'entrée unique de l'application. Elle se compose d'un panneau latéral de présentation de la plateforme et d'un formulaire d'authentification demandant le nom d'utilisateur et le mot de passe. En cas d'échec, un message d'erreur générique s'affiche, sans préciser si le nom d'utilisateur saisi correspond ou non à un compte existant, conformément au principe de non-énumération. Un lien « Mot de passe oublié ? » donne accès au formulaire de demande de réinitialisation, dont le second écran permet de saisir un nouveau mot de passe après réception du lien sécurisé transmis par courriel.

Un formulaire d'inscription est également accessible depuis cette même page, permettant aux fournisseurs de s'auto-inscrire en renseignant leur raison sociale, numéro ICE, catégorie d'activité et coordonnées. L'inscription crée un compte en statut « en attente de validation » : le fournisseur ne pourra se connecter qu'après approbation par l'administrateur.

3.6.2 Interface d'administration

L'interface d'administration se présente sous forme d'un tableau de bord avec une barre de navigation latérale donnant accès aux différentes sections :

- **Tableau de bord** : indicateurs clés de la plateforme (nombre de fournisseurs par statut, appels d'offres en cours, candidatures en attente, factures en traitement).
- **Gestion des fournisseurs** : liste paginée et filtrable de l'ensemble des fournisseurs référencés, avec pour chacun les actions de validation, rejet, suspension et réintégration.

- **Gestion des utilisateurs** : création, modification, activation/désactivation des comptes, attribution des rôles.
- **Journal d'audit** : consultation chronologique de toutes les actions enregistrées par le système, filtrables par type, par acteur et par période.

3.6.3 Interface du service achat

L'interface du service achat est dédiée à la gestion opérationnelle des marchés :

- **Mes appels d'offres** : liste des appels d'offres créés par l'agent connecté, avec les actions de modification, publication, clôture et annulation.
- **Candidatures** : liste des candidatures reçues, avec pour chacune la proposition technique, le montant proposé et les actions d'acceptation ou de rejet.
- **Fournisseurs** : consultation des fiches fournisseurs et de leurs documents administratifs, avec les actions de validation et de rejet.
- **Factures** : traitement des factures soumises par les fournisseurs (validation, paiement, rejet).

3.6.4 Interface fournisseur

L'interface fournisseur est un espace dédié accessible après connexion et validation du compte :

- **Mon profil** : consultation et modification de la fiche fournisseur (uniquement en statut brouillon), avec bouton de soumission pour validation.
- **Mes documents** : téléversement et suivi des pièces justificatives, avec indication du statut de validation de chaque document et de sa date d'expiration.
- **Appels d'offres** : consultation des appels d'offres publiés et ouverts, avec possibilité de soumettre une candidature (proposition technique et montant proposé).
- **Mes candidatures** : suivi de l'état de chaque candidature (soumise, acceptée, rejetée) avec le commentaire éventuel du service achat.
- **Mes factures** : émission de factures liées aux candidatures acceptées et suivi de leur traitement (en attente, validée, payée, rejetée).

3.7 Conclusion

Voilà, dans ses grandes lignes, comment le portail fournisseur a été construit : les technologies retenues pour le back-end et le front-end, l'architecture hexagonale mise en place, les mécanismes de sécurité DGSSI présents à chaque niveau de l'application, le fonctionnement du module de messagerie, et un tour des interfaces exploitables par chacun des trois profils. Tout ce qui avait été décidé au chapitre 2 se retrouve, concrètement, dans ce code.

CHAPITRE 4

Conclusion générale et perspectives

4.1 Introduction

Ce dernier chapitre revient sur l'ensemble du travail accompli durant ce stage de fin d'année : un bilan des objectifs atteints, les difficultés rencontrées en cours de développement, les compétences que j'en retire, et quelques pistes pour la suite, si le projet devait être repris et déployé en production.

4.2 Bilan du travail réalisé

Le portail fournisseur, tel que présenté dans ce rapport, répond à l'ensemble des objectifs fixés au chapitre 1. L'application permet désormais :

- l'auto-inscription sécurisée des fournisseurs avec validation obligatoire du numéro ICE et approbation administrative préalable à l'activation du compte ;
- la gestion centralisée des dossiers fournisseurs et de leurs pièces justificatives, avec un cycle de validation documenté et traçable ;
- la publication et la gestion complète des appels d'offres, depuis la création en brouillon jusqu'à la clôture, avec soumission et instruction des candidatures ;
- le suivi de la facturation liée aux marchés attribués, avec un circuit de validation à plusieurs étapes ;
- un contrôle d'accès fin, articulé autour de trois profils d'utilisateurs aux prérogatives strictement délimitées et systématiquement vérifiées côté serveur ;
- une conformité validée avec les exigences de cybersécurité applicative de la DGSSI : authentification stateless, hachage BCrypt, jetons à usage unique, limitation de débit, en-têtes de sécurité HTTP, non-énumération des comptes et journalisation d'audit inaltérable ;
- un code source maintenable et structuré selon les principes de la Clean Architecture, avec une séparation rigoureuse entre domaine métier et détails d'infrastructure.

Ce bilan montre qu'une solution sur mesure, développée dans le cadre d'un stage de fin d'année, peut répondre de manière concrète et opérationnelle aux besoins d'un organisme public en matière de gestion des fournisseurs et de conformité réglementaire.

4.3 Difficultés rencontrées

Ce projet n'a pas été sans difficultés, et leur résolution représente une bonne partie de ce que j'en retire.

La première difficulté, et sans doute la plus structurante, a été la **conception d'un modèle de droits d'accès cohérent**, applicable de manière homogène à la fois côté serveur (sécurité réelle) et côté client (expérience utilisateur). Il a fallu veiller à ce que chaque restriction visible dans l'interface trouve une contrepartie stricte et systématique dans le contrôle d'accès du serveur, afin d'éviter toute incohérence entre ce qui est affiché à l'utilisateur et ce qui est effectivement autorisé.

Une autre difficulté a porté sur la **mise en place du circuit de validation des comptes fournisseurs**. L'inscription crée un compte en statut désactivé : le fournisseur reçoit un courriel de confirmation contenant un mot de passe temporaire uniquement après validation administrative. Ce flux asynchrone, impliquant deux

acteurs (le fournisseur et l'administrateur), a nécessité une attention particulière pour garantir la cohérence des états à chaque étape et pour s'assurer que le mot de passe temporaire est bien changé dès la première connexion.

Une troisième difficulté a porté sur la **fiabilisation de l'envoi de courriels**. La configuration SMTP, la gestion des erreurs d'envoi (serveur inatteignable, identifiants invalides) et l'envoi asynchrone sans blocage de la requête HTTP ont demandé plusieurs itérations avant d'aboutir à un mécanisme fiable.

Enfin, la **mise en place de l'architecture hexagonale** a représenté un effort supplémentaire par rapport à une architecture en couches classique. La discipline consistant à ne jamais importer de classe Spring dans le domaine, à définir des ports et des adaptateurs pour chaque interaction avec l'extérieur, demande une rigueur constante ; mais le gain en termes de testabilité et de maintenabilité justifie pleinement cet investissement.

4.4 Compétences acquises

Ce stage a été l'occasion de mettre en pratique de nombreuses compétences abordées durant ma formation, et d'en acquérir de nouvelles :

- Maîtrise avancée du développement d'API REST sécurisées avec **Spring Boot 3** et **Spring Security 6**.
- Application pratique des principes de la **Clean Architecture** et du **Domain-Driven Design (DDD)** : entités riches, objets de valeur, ports et adaptateurs.
- Expertise concrète sur les normes de cybersécurité nationale (**DGSSI/DNSSI**) et internationales (**OWASP**) : hachage BCrypt, jetons à usage unique, en-têtes de sécurité HTTP, non-énumération, limitation de débit.
- Approfondissement de la modélisation UML et de la conception de bases de données relationnelles normalisées.
- Découverte de la gestion des migrations de schéma de base de données avec **Flyway**.
- Compréhension du fonctionnement des marchés publics marocains et des contraintes réglementaires applicables aux organismes d'importance vitale.

4.5 Perspectives d'évolution

Plusieurs axes d'amélioration me semblent intéressants pour une éventuelle mise en production :

- **Application mobile** : le découplage strict entre le back-end et le front-end, matérialisé par l'API REST, permettrait d'envisager le développement d'une application mobile complémentaire, notamment destinée aux fournisseurs en déplacement.
- **Authentification multi-facteurs (2FA)** : l'ajout d'un second facteur d'authentification (OTP par SMS ou par application) renforcerait la sécurité des comptes, conformément aux recommandations renforcées de la DGSSI pour les opérations critiques.
- **Signature électronique qualifiée** : l'intégration d'un mécanisme de signature électronique conforme à la loi 53-05 permettrait de dématérialiser complètement les documents contractuels.
- **Notifications en temps réel** : le remplacement du mécanisme actuel de rafraîchissement par une communication en temps réel (par exemple via WebSocket) améliorerait la réactivité perçue de l'application.
- **Module de reporting** : l'ajout de tableaux de bord analytiques avec des graphiques d'évolution dans le temps (volume d'appels d'offres, taux de validation, délais de traitement) fournirait un outil de pilotage aux décideurs.
- **Tests automatisés** : la mise en place d'une suite de tests unitaires et d'intégration automatisés renforcerait la confiance lors des évolutions futures de l'application.

4.6 Conclusion

Au terme de ce stage de fin d'année au sein de l'ONEE, j'ai conçu et réalisé un portail fournisseur complet et sécurisé, qui répond à une problématique concrète : celle de la dématérialisation et de la sécurisation des relations fournisseurs dans un organisme d'importance vitale, en conformité avec les exigences de la DGSSI.

La démarche suivie — analyse rigoureuse des besoins, conception détaillée fondée sur une modélisation UML cohérente et sur les principes de la Clean Architecture, puis réalisation progressive et testée à chaque étape — m'a permis d'aboutir à une solution opérationnelle, couvrant l'ensemble du cycle de vie d'un fournisseur : de son inscription au portail jusqu'au règlement de ses factures, en passant par la validation de ses pièces administratives, sa candidature aux appels d'offres et l'instruction de ses dossiers par le service achat.

Ce projet ne s'est pas arrêté aux fonctionnalités de gestion classiques attendues d'un tel outil : il m'a aussi permis d'explorer des dimensions plus avancées du développement logiciel professionnel, notamment la sécurisation rigoureuse d'une API REST selon les standards nationaux et internationaux, la conception d'un mécanisme d'authentification robuste et la mise en place d'une architecture logicielle propre et maintenable.

Ce travail constitue, à mes yeux, une synthèse représentative des compétences développées au cours de ma formation à l'EMSI, et une base solide pour ma carrière professionnelle d'ingénieur. J'espère que les perspectives d'évolution évoquées plus haut donneront, un jour, l'occasion de pousser ce portail encore plus loin et de le déployer en production au service des fournisseurs de l'ONEE.

Bibliographie et webographie

- [1] **DGSSI**, *Directive Nationale de la Sécurité des Systèmes d'Information (DNSSI)*, Royaume du Maroc, Administration de la Défense Nationale.
- [2] **Spring.io**, *Spring Boot Reference Documentation (v3.x)*, <https://docs.spring.io/spring-boot/>
- [3] **Spring.io**, *Spring Security Reference Guide*, <https://docs.spring.io/spring-security/>
- [4] **OWASP Foundation**, *OWASP Top Ten Web Application Security Risks*, <https://owasp.org/www-project-top-ten/>
- [5] **Robert C. MARTIN**, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- [6] **IETF**, *RFC 7519: JSON Web Token (JWT)*, <https://datatracker.ietf.org/doc/html/rfc7519>
- [7] **Oracle Corporation**, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] **Flyway**, *Flyway Database Migrations Documentation*, <https://flywaydb.org/documentation/>
- [9] **Niels PROVOS**, **David MAZIÈRES**, *A Future-Adaptable Password Scheme*, USENIX Annual Technical Conference (présentation de l'algorithme de hachage BCrypt).
- [10] **Mozilla Developer Network**, *Documentation des standards HTML, CSS et JavaScript*, <https://developer.mozilla.org/fr/>
- [11] **Pascal ROQUES**, *UML 2 par la pratique : études de cas et exercices corrigés*, Eyrolles.
- [12] **Roy T. FIELDING**, *Architectural Styles and the Design of Network-based Software Architectures*, thèse de doctorat, University of California, Irvine.

Annexes

Annexe A : Liste complète des tables de la base de données

Table	Contenu
users	Comptes utilisateurs (administrateurs, agents du service achat, fournisseurs)
suppliers	Fiches fournisseurs (raison sociale, ICE, catégorie, statut, coordonnées)
supplier_documents	Pièces justificatives des fournisseurs (type, fichier, statut de validation, date d'expiration)
supplier_evaluations	Évaluations de fournisseurs par les agents (score global, date, commentaire)
evaluation_criteria_scores	Scores détaillés par critère d'évaluation
appel_offres	Appels d'offres (titre, description, catégorie, budget, dates, statut)
demandes	Candidatures des fournisseurs aux appels d'offres (proposition technique, montant, statut)
factures	Factures émises par les fournisseurs (référence, montant, dates, statut)
audit_log	Journal d'audit inaltérable (acteur, action, cible, IP, résultat, détail)
password_reset_tokens	Jetons de réinitialisation de mot de passe (empreinte SHA-256, usage unique, durée limitée)
notifications	Notifications internes (destinataire, type, message, statut de lecture)

Annexe B : Extrait du script de création de la table suppliers

```
CREATE TABLE suppliers (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_name VARCHAR(255) NOT NULL,
  ice VARCHAR(15) NOT NULL UNIQUE,
  commercial_register VARCHAR(50) NULL,
  tax_identifier VARCHAR(50) NULL,
  address VARCHAR(500) NULL,
  contact_person VARCHAR(255) NULL,
  contact_email VARCHAR(255) NULL,
  contact_phone VARCHAR(30) NULL,
  category VARCHAR(20) NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'DRAFT',
  registered_by_user_id BIGINT NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NULL ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (registered_by_user_id) REFERENCES users(id),
  INDEX idx_suppliers_status (status),
  INDEX idx_suppliers_category (category)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

Annexe C : Extrait du script de création de la table appel_offres

```
CREATE TABLE IF NOT EXISTS appel_offres (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  titre       VARCHAR(500) NOT NULL,
  description  TEXT NOT NULL,
  categorie    VARCHAR(50) NOT NULL,
  budget_estime DECIMAL(15,2) NULL,
  date_publication DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  date_cloture DATETIME NOT NULL,
  status       VARCHAR(30) NOT NULL DEFAULT 'BROUILLON',
  published_by_user_id BIGINT NOT NULL,
  created_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   DATETIME NULL ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (published_by_user_id) REFERENCES users(id),
  INDEX idx_ao_status (status),
  INDEX idx_ao_categorie (categorie),
  INDEX idx_ao_date_cloture (date_cloture)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

Annexe D : Glossaire

Terme	Définition
ICE	Identifiant Commun de l'Entreprise, numéro à 15 chiffres attribué par l'administration fiscale marocaine à toute personne morale exerçant une activité économique.
Appel d'offres	Procédure de mise en concurrence par laquelle un organisme public sollicite des propositions de fournisseurs pour la réalisation de travaux, la fourniture de biens ou la prestation de services.
Candidature	Réponse d'un fournisseur à un appel d'offres, comprenant une proposition technique et un montant proposé.
Jeton (token)	Donnée signée permettant de matérialiser l'authentification d'un utilisateur sans transmettre à nouveau son mot de passe.
Hachage	Transformation irréversible d'une donnée (ici, un mot de passe) en une empreinte de taille fixe, utilisée pour en vérifier l'exactitude sans jamais la stocker en clair.
Principe du moindre privilège	Principe de sécurité selon lequel un utilisateur ne doit disposer que des droits strictement nécessaires à l'accomplissement de ses tâches.
Non-énumération	Principe de sécurité visant à empêcher un attaquant de déduire la liste des comptes existants à partir des réponses de l'application.
Clean Architecture	Paradigme d'architecture logicielle où le domaine métier est indépendant de tout framework, base de données ou interface utilisateur.
Port	Interface définie par la couche domaine, spécifiant un contrat que la couche infrastructure doit implémenter (architecture hexagonale).
Adaptateur	Implémentation concrète d'un port, reliant le domaine métier à une technologie spécifique (JPA, SMTP, etc.).